Writing skills *for Claude Code* agents.

A practical guide based on the QA agent pipeline in agent-demo-app

Local Content Review Platform · DemoApp
October 2026

centric

Agenda
Table of contents

1. The .claude folder and the QA agent pipeline
2. Agents and skills: who does what
3. How to write a good skill (common patterns)
4. Adding MCP servers to Claude Code
5. Calling an agent: @agent-name task
6. Permissions for files and MCP servers
7. Recommendations for this repository

centric

The .claude *folder.*

Where agents and skills live in this repository, and how the five QA agents hand work to each other.

centric

Structure of automation/.claude

```
automation/
├── .claude/
│   ├── agents/                # one .md per agent
│   │   ├── qa-planner.md
│   │   ├── qa-explorer.md
│   │   ├── qa-test-case-writer.md
│   │   ├── qa-bug-reporter.md
│   │   └── qa-automator.md
│   └── skills/                # one folder per skill
│       └── qa-planner/
│           ├── SKILL.md       # entry point
│           ├── references/    # loaded on demand
│           │   ├── ado-qa-task.md
│           │   ├── feature-map.md
│           │   ├── test-data-constraints.md
│           │   └── example.md
│           └── scripts/
│               └── check-feature-map.mjs
├── .mcp.json                  # project MCP servers
├── lessons/<agent>-lessons.md  # learned rules
├── archive/<PBI_ID>/          # outputs (git-ignored)
├── tools/qa-login.mjs         # sign-in helper
└── fe_automation/             # Playwright project
```

agents/<name>.md

Frontmatter only: name, description, tools, model, effort, permissionMode and the skill it preloads. It decides **what the agent may use**.

skills/<name>/SKILL.md

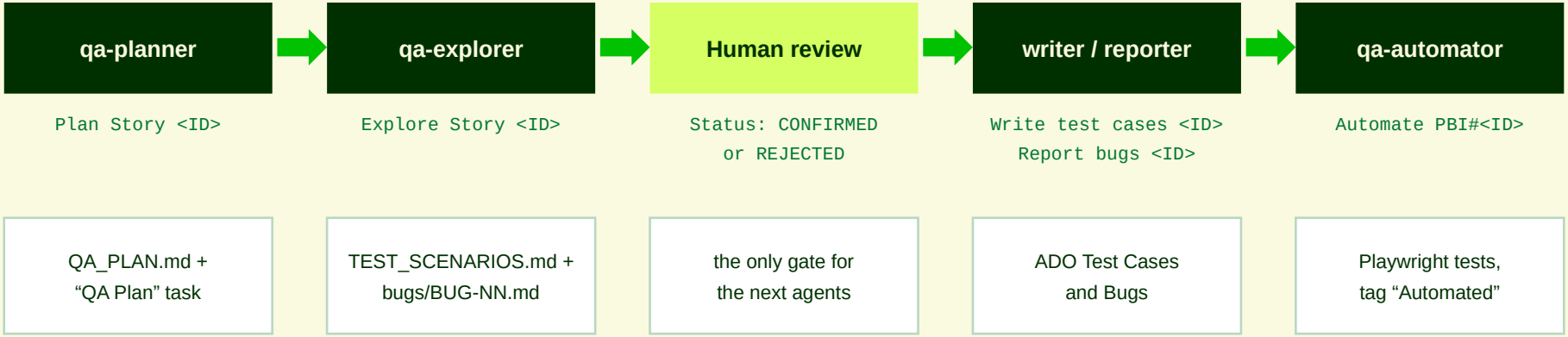
The playbook: inputs, tools, numbered workflow, guardrails, lessons, example. It decides **how the agent works**. References and scripts sit next to it.

Start Claude Code from automation/

Project agents, skills and .mcp.json are discovered from the working directory. The app's CLAUDE.md in the parent folder is still loaded, which is why the skills say “already in your context”.

centric

The QA agent pipeline



Shared workspace: every agent reads and writes `archive/<PBI_ID>/`, always keyed by the PBI id, even when started on a Task.
Stable ids: scenarios are `FE1…` / `BE1…` and are never renumbered; downstream agents match by id, never by wording.
Board tracking: each agent owns exactly one Azure DevOps work item type it may touch (its QA task, Test Cases, Bugs or tags).
Idempotent: created ids are written back into the files (`ado_bug:`, `Created:`), so a re-run never duplicates work.

centric

Supporting files around .claude

File / folder	Purpose	Rule
`.mcp.json`	Project MCP servers: postgres, swagger, azure-devops, chrome-devtools-mcp, playwright	Shared through git; secrets via ${ENV_VAR}
`lessons/<agent>-lessons.md`	Rules learned from human corrections; read at step 0 of every run	Append only when a human corrects the agent
`archive/<PBI_ID>/`	QA_PLAN.md, TEST_SCENARIOS.md, bugs/ — the hand-off between agents	Git-ignored; reviewed by a human
`.secrets/qa.env`	QA account credentials used by the sign-in helper	Git-ignored; agents never read it
`tools/qa-login.mjs`	Signs a QA account in to the QA Chrome without exposing the password or token	Only Bash command most agents may run
`skills/*/scripts/*.mjs`	Deterministic checks, e.g. check-feature-map.mjs	Prefer a script over “check by reading”
`fe_automation/`	Playwright + TypeScript project with page objects and SQL test data	Only qa-automator writes here

centric

Agents *and skills.*

An agent definition grants capabilities. A skill describes the procedure. Keep the two separate.

centric

Anatomy of an agent file

```
# .claude/agents/qa-planner.md
---
name: qa-planner
description: Analyzes the requirements
  and acceptance criteria of a PBI from
  Azure DevOps, creates a QA plan for the
  following agents and tracks it in the
  PBI's "QA Plan" task.
tools:
  - Read
  - Write
  - Edit
  - Grep
  - Skill
  - mcp__azure-devops
  - mcp__swagger
  - mcp__postgres
model: opus
effort: high
permissionMode: acceptEdits
skills:
  - qa-planner
---
```

Field	What it does
`name`	Id used by @agent-<name>; matches the skill name
`description`	When Claude should delegate to it; state inputs and outputs
`tools`	Allowlist. Omit it and the agent inherits every tool
`mcp__<server>`	All tools of one MCP server (see Permissions)
`model` / `effort`	Model alias and reasoning effort for this agent
`permissionMode`	acceptEdits: file edits run without a prompt
`skills`	Skills preloaded into the agent's context
body (optional)	System prompt. Empty here: the skill holds the procedure

centric

Agent vs. skill

Agent — `.claude/agents/<name>.md`

- Runs in its own context window
- Hard limits: tools, model, permission mode
- Called with @agent-<name> or by delegation
- Short: frontmatter, optional system prompt

Use it for a role with its own tool set (planner, explorer, automator)

Skill — `.claude/skills/<name>/SKILL.md`

- Loaded on demand by the main session or an agent
- Soft limits: the rules the model follows
- Called with /<name> or picked by its description
- Long: workflow, guardrails, references, scripts

Use it for a repeatable procedure that several agents or sessions may need

centric

Writing *good skills.*

The common elements extracted from qa-planner, qa-explorer, qa-test-case-writer, qa-bug-reporter and qa-automator.

centric

The common SKILL.md skeleton (1/2)

#	Section	Purpose	In this repo
1	Frontmatter	`name` + a `description` that says what it reads, writes and tracks	All five skills; qa-automator's names tag, folder, task
2	Title + role	One paragraph: the agent's place in the pipeline (before / after whom)	“The step after qa-test-case-writer…”
3	System under test	URLs, tracker org/project; what is already in context and must not be re-read	API :5080, portals :5173/:5174, DemoApp
4	References table	File → when to load it. Keeps SKILL.md short (progressive disclosure)	references/ado-qa-task.md — “Steps 0 and 11”
5	Inputs	Exact commands, variants (subset, dry run), Task id → parent PBI	`Report bugs <ID> BUG-02 dry run`

centric

The common SKILL.md skeleton (2/2)

#	Section	Purpose	In this repo
6	Tools / MCP servers	Per tool: what may be read, what may be written, and an explicit “nothing else” list	azure-devops: write only the QA task
7	Workflow	Numbered steps 0…N. Step 0 = board / lessons; last = report + close the task	qa-explorer: 0 Board … 11 Close on the board
8	Guardrails	Non-negotiable rules that override everything else in the skill	Credentials, synthetic data, black-box
9	Lessons	Read at the start; append only when a human corrects the agent	lessons/qa-*-lessons.md with a fixed format
10	Example	One realistic run: input, what it finds, what it writes, the final comment	“User: Report bugs 292.” → 2 created, 1 skipped

centric

Frontmatter and context

Write the description as a contract

- What it reads (Test Cases tagged “Should be automated”)
- What it produces (Playwright tests in fe_automation)
- Where it tracks work (the PBI's “QA Automation” task)
- Claude picks skills and agents by this text: be specific

Keep name identical in the agent file, the skill folder and the frontmatter

Give the agent its world in a few lines

- Component URLs and the tracker (org, project)
- Which docs are the source of truth, and which win on conflict (AC > docs)
- Say what is already loaded: “CLAUDE.md is in your context — do not Read it again”

Large files: grep the heading, then Read with offset / limit (docs/api.md ≈ 10k tokens)

centric

Progressive disclosure: references and scripts

```
## References (load only when the step needs it)

| File                          | When                       |
|-------------------------------|----------------------------|
| references/ado-qa-task.md     | Steps 0 and 11 - board     |
| references/environment.md     | Step 1 - URLs, sessions    |
| references/fixtures.md        | Step 4 - run id, cleanup   |
| references/output-templates   | Steps 7-9 - templates      |
```

SKILL.md stays at ~120–200 lines; details move to `references/`.

Every reference is tied to **the step that needs it**.

Share instead of copying: `../qa-planner/references/feature-map.md` is reused by explorer and automator.

Templates for outputs (bug file, test case, plan) live in references, so the format never drifts.

scripts/ for deterministic checks

`check-feature-map.mjs` verifies every doc and api.md section named in the feature map still exists.

Exit codes: 0 in sync · 1 stale entries · 2 cannot read inputs.

tools/ to keep secrets away from the model

`qa-login.mjs` reads `.secrets/qa.env` and signs in through the DevTools protocol.

It prints a one-line result — never the password or the session token.

centric

Inputs: exact commands, predictable variants

Agent	Command
qa-planner	`Plan Story <ID>` · `Plan Task <ID>`
qa-explorer	`Explore Story <ID>` · `Explore Task <ID>`
qa-test-case-writer	`Write test cases <ID> [FE1 FE7] [dry run]`
qa-bug-reporter	`Report bugs <ID> [BUG-02 BUG-05] [dry run]`
qa-automator	`Automate PBI#<ID> [TC <id> …] [dry run]`

Same grammar everywhere: verb + work item id.

Subset: list ids to limit the run (still respecting the status gate).

Dry run: read and plan only; show what would be written; no board changes.

Task id accepted: the agent resolves the parent PBI and says so.

No credentials as input: the agent signs in itself through the helper; if setup fails it asks a human.

centric

Tools section: scope every tool

```
## Tools / MCP servers
- `azure-devops` - read the PBI, its Test Cases and
  the parent Feature. Write ONLY: the PBI's QA
  Automation task and System.Tags of the Test Cases
  automated in this run. Nothing else.
- `postgres` - read-only. Schemas before writing a
  query, leftover check after the run.
- `Bash` - only: node tools/qa-login.mjs ... and
  npm --prefix fe_automation test -- <args>.
- `Write` / `Edit` - only inside fe_automation/
  and the lessons file.
```

Black-box rule (planner, explorer): expected results come from Azure DevOps, Swagger, CLAUDE.md and docs/ — never from source code, PRs, commits or git history.

For **each** tool state the read scope and the write scope.

Name the allowed MCP tool calls (`wit_work_item get`, `wit_work_item_write update`).

Add a **negative list**: no `repo_*`, `pipelines_*`, `search_code`, attachments.

Say what to do when a tool fails: “auth error → ask the human to reconnect (/mcp)”.

Say when to **stop**: “Stop if azure-devops or swagger MCP is not available”.

centric

Workflow: numbered, bounded, verifiable

0	**Board + lessons**	Resolve the PBI, find or create the agent's QA task, set it in progress, read lessons.
1–N	**One action per step**	Each step names the reference it needs and its stop condition (“none → say so, stop”).
Verify	**Prove the result**	Leftover check with postgres; tests must actually run; re-read output against the guardrails.
Limit	**Bounded retries**	At most 3 fix attempts per failing test, then leave it failing and report it.
End	**Report + close**	Summary to the human and in the task Description; close only if everything succeeded.

centric

Guardrails that every skill repeats

One write target

Each agent may create or change exactly one kind of work item. Everything else is read-only.

Human gate

Only items marked CONFIRMED or ACCEPTED move on. Never PENDING, REJECTED or unclear.

Credentials

Never read .secrets/ or .env, never print tokens, invitation codes only as last 4 characters.

Synthetic data

Run-tagged @example.test / example.com data; never touch seeded or demo records.

Transcribe, don't invent

Every fact comes from the source file or work item; gaps go to Open Questions.

Hands off the app

Never edit backend/, frontend/, docs/, CLAUDE.md; never reset the stack; never commit.

centric

Hand-offs and lessons

Hand-offs between agents

- Files in archive/<PBI_ID>/ with a fixed format
- A `### Handoff` section: what to verify first, run order
- Flags with exact spelling: `automate · regression · testcase`
- Write-back of created ids makes every run idempotent
- A duplicate check in the tracker before every create

```
# lessons/qa-planner-lessons.md
## YYYY-MM-DD - <short title>
- Lesson: <the rule>
- Why: <what went wrong or
  what the user said>
- How to apply: <when and how
  it changes the plan>
```

Read at step 0. A lesson overrides the general guidance, **never the guardrails**.

Only human corrections are added — never lessons the agent inferred itself.

centric

A skill template you can copy

```
---
name: my-skill
description: <What it reads>, <what it
  writes>, <where it tracks the work>.
---

# My Skill
<Role: the step after X, before Y.>

**System under test:** <URLs, tracker>.

## References (load only when needed)
| File                   | When        |
|------------------------|-------------|
| references/<file>.md   | Step <n>    |

## Inputs
- `<Verb> <ID>` - <default scope>
- `<Verb> <ID> dry run` - no writes
```

```
## Tools / MCP servers
- `<server>` - read: ... write ONLY: ...
- Nothing else: no <tool families>.

## Workflow
0. Lessons + board: resolve, in progress.
1. <step> (references/<file>.md)
2. <step> - none found -> say so, stop.
N. Report to the human; close the task.

## Guardrails
- <the only thing it may write>
- No credentials, synthetic data only.

## Lessons
Append to lessons/my-skill-lessons.md
only when a human corrects you.

## Example
User: "<Verb> 123." -> <what happens>
```

centric

Checklist before you ship a skill

1. Description says **what it reads, writes and tracks** — and matches the agent's description.
2. SKILL.md under ~200 lines; detail moved to `references/`, each tied to a step.
3. Inputs are exact commands, with subset and **dry run** variants.
4. Every tool has a read scope, a write scope and a negative list.
5. Workflow is numbered, with stop conditions, a verification step and bounded retries.
6. Guardrails cover: the single write target, credentials, data, the human gate.
7. Lessons file exists with its format; one realistic example at the end.
8. The agent's `tools:` list contains **only** what the skill needs.

centric

Commands *and configuration.*

Adding MCP servers, calling an agent, and granting permissions to files and MCP tools.

centric

1. Adding MCP servers — .mcp.json

```
// automation/.mcp.json (shortened)
{
  "mcpServers": {
    "azure-devops": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@azure-devops/mcp",
               "demo-app-gabi",
               "--authentication", "pat"],
      "env": {
        "PERSONAL_ACCESS_TOKEN":
          "${AZURE_DEVOPS_MCP_TOKEN}"
      }
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest"]
    }
  }
}
```

Project scope: `.mcp.json` in the folder you start Claude from; committed and shared with the team.

Secrets: use `${VAR}` expansion — the PAT comes from `AZURE_DEVOPS_MCP_TOKEN`, never from the file.

Windows: wrap npx with `"command": "cmd", "args": ["/c", "npx", …]` (as chrome-devtools-mcp does).

First start: Claude asks you to approve project servers; check them with `/mcp`.

Tool names become `mcp__<server>__<tool>`, e.g. `mcp__azure-devops__wit_work_item`.

centric

1. Adding MCP servers — the claude mcp CLI

```
# stdio server, shared with the team (.mcp.json)
claude mcp add --scope project playwright \
  -- npx -y @playwright/mcp@latest

# with an environment variable
claude mcp add --scope project swagger \
  -e SPEC_URL=http://localhost:5080/swagger/v1/swagger.json \
  -- node ./mcps/swagger/dist/index.js

# remote HTTP server
claude mcp add --transport http <name> <https-url>
```

```
claude mcp list            # all servers + status
claude mcp get playwright  # details of one server
claude mcp remove playwright
/mcp                       # inside a session: status, re-auth
```

Scope	Stored in / visible to
`local` (default)	~/.claude.json · only you, this project
`project`	.mcp.json · everyone who clones the repo
`user`	~/.claude.json · only you, all projects

Everything after `--` is the server command.

Use `project` for the QA servers so every agent run uses the same configuration.

centric

2. Calling an agent

```
@agent-qa-planner Plan Story 112
@agent-qa-explorer Explore Story 112
@agent-qa-test-case-writer Write test cases 112 dry run
@agent-qa-bug-reporter Report bugs 112 BUG-01 BUG-03
@agent-qa-automator Automate PBI#50
```

Type `@`, pick the agent from the list (shown as **agent-<name>**), then write the task in the skill's input format.

The mention guarantees that this agent runs; it works in its own context and returns a report.

Natural language also works: “Use the qa-planner agent to plan Story 112”.

Command	What it does
`@agent-<name> <task>`	Run that agent with a task
`/agents`	List, create, edit agents and their tools
`/<skill-name> <args>`	Run a skill in the current session, e.g. /qa-planner
`/mcp`	Check that the agent's MCP servers are connected

Tip: run agents one at a time along the pipeline, and do the human review (CONFIRMED / REJECTED) between explorer and writer/reporter.

centric

3. Permissions — how this repo does it

Agent tools: allowlist	Hard limit enforced by Claude Code. qa-bug-reporter gets Read, Glob, Grep, Edit, Skill, mcp__azure-devops — no Write, no Bash, no browser.
MCP granularity	`mcp__azure-devops` = every tool of the server. `mcp__azure-devops__wit_work_item` = one tool. The repo grants whole servers.
permissionMode	All five agents use `acceptEdits`: edits run without a prompt; Bash and MCP calls still follow the normal rules.
Skill rules (soft)	Narrow the scope in prose: “Write/Edit only under archive/<PBI_ID>/”, “Bash only node tools/qa-login.mjs”.
Files kept out of reach	`.gitignore` excludes archive/ and .secrets/; qa-login.mjs reads the secrets so the model never has to.

centric

3. Permissions — settings.json rules

```
// automation/.claude/settings.json
{
  "permissions": {
    "allow": [
      "Bash(node tools/qa-login.mjs:*)",
      "Bash(npm --prefix fe_automation test:*)",
      "Read(../docs/**)",
      "mcp__swagger",
      "mcp__azure-devops__wit_work_item"
    ],
    "ask": [
      "mcp__azure-devops__wit_work_item_write"
    ],
    "deny": [
      "Read(./.secrets/**)",
      "Read(./fe_automation/.env)",
      "Edit(../backend/**)",
      "mcp__azure-devops__repo_create_branch"
    ]
  },
  "enabledMcpjsonServers": ["azure-devops", "swagger"]
}
```

Order: deny > ask > allow. A deny rule also blocks agents with `acceptEdits`.

Files: `Read(path)` and `Edit(path)` with gitignore-style globs (`**`).

Bash: `Bash(cmd:*)` allows a command prefix.

MCP: `mcp__server` (all tools) or `mcp__server__tool` (one tool).

Where: `.claude/settings.json` (team, committed) · `.claude/settings.local.json` (personal, git-ignored) · `~/.claude/settings.json` (all projects).

Manage interactively with `/permissions`.

centric

Recommendations for this repository

Turn soft rules into hard ones

The skills forbid reading .secrets/ and .env, but nothing enforces it. Add deny rules in .claude/settings.json.

Narrow MCP grants

Agents get all of mcp__azure-devops, including repo, pipeline and wiki writes. List only the tools each skill names.

Keep .mcp.json portable

The postgres URL is inline and the swagger/postgres servers use C:/Users/... paths. Use ${DATABASE_URL} and repo-relative paths.

Use disallowedTools for exceptions

Keep a whole-server grant but remove the risky tools of that server for one agent.

centric

Questions?

We’re happy to discuss

centric


centric